



NexaLearn — AI-Powered E-Learning Platform Backend

A Graduation Project Report Submitted to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the requirements of the degree of
Bachelor of Science in Computer Engineering.

Presented by

Youssef Bahy Youssef

Ahmed Kamal

Ali Afifi

Rufida Kassem

Supervised by

Dr. Lydia Wahid

Department of Computer Engineering

Academic Year: 2025/2026

All rights reserved. This report may not be reproduced in whole or in part,
by photocopying or other means, without the permission of the authors/department.

Abstract

The global shift toward digital education has intensified demand for backend platforms that can orchestrate authentication, structured course delivery, secure payments, media processing, automated assessment, and real-time learner engagement within a single cohesive system. Existing solutions often address these concerns in isolation: commercial Learning Management Systems (LMS) provide turnkey features but limit customisation and data ownership; legacy open-source platforms rely on monolithic designs that resist modern scalability requirements; and generic web application templates omit domain-specific concerns such as enrollment state machines, payment webhook idempotency, video transcoding pipelines, and AI-assisted content generation. This fragmentation leaves educators and engineering teams without a complete, production-ready reference architecture for end-to-end e-learning operations.

This graduation project addresses that gap by designing and implementing **NexaLearn**, a full-featured e-learning backend built with NestJS and TypeScript. The primary objective is to deliver a modular monolith organised according to Domain-Driven Design (DDD), comprising more than fifteen bounded contexts—including Authentication, Courses, Enrollment, Payments (Stripe), Media (Mux video), AI Pipeline (quiz generation via Whisper transcription and large language models), Assessment, Discussion, Progress, Reviews, Certificates, Search, Streak, Notifications, and Instructor Analytics—each encapsulated with clear domain boundaries, application use cases, and infrastructure adapters.

The adopted approach follows Clean Architecture and CQRS-lite separation of commands and queries, complemented by an event-driven integration model and a transactional outbox pattern to guarantee reliable, at-least-once event delivery with exactly-once side effects across contexts. Security is enforced through JSON Web Token (JWT) authentication with refresh-token rotation, role-based access control (RBAC), and HMAC-signed service-to-service calls for internal pipeline communication. Persistence is managed with PostgreSQL and the Prisma ORM; Redis supports caching, session management, and rate limiting; Stripe handles checkout and refund flows; Mux manages direct-to-cloud video upload and transcoding; and AWS S3 stores supplementary assets. The system is containerised with Docker for reproducible deployment.

The resulting platform exposes a comprehensive REST API covering enrollment lifecycles, lesson progress projection, quiz attempts, discussion moderation, certificate generation, full-text course search, and instructor analytics dashboards. Inte-

gration testing with Jest and Testcontainers validates critical paths including concurrent outbox processing, payment activation flows, and access-control enforcement. NexaLearn demonstrates that a rigorously structured NestJS backend can unify the functional breadth of a modern e-learning marketplace while preserving maintainability, testability, and operational readiness.

Keywords: E-Learning, NestJS, Domain-Driven Design, Clean Architecture, Transactional Outbox, Stripe, Mux, Artificial Intelligence, PostgreSQL

الملخص

يشهد قطاع التعليم الإلكتروني نمواً متسارعاً يفرض الحاجة إلى منصات خلفية متكاملة قادرة على إدارة المصادقة، ومحتوى الدورات، ومعالجة الفيديو، والمدفوعات، والاختبارات المولدة بالذكاء الاصطناعي، وتفاعل المتعلمين في نظام واحد متماسك. ومع ذلك، تفتقر الحلول المتاحة حالياً — سواء المنصات التجارية المغلقة أو الأنظمة مفتوحة المصدر القديمة أو القوالب العامة — إلى التكامل الكامل بين هذه المتطلبات، مما يحد من قابلية التوسع والتخصيص والجاهزية للإنتاج.

يهدف هذا المشروع إلى تصميم وتنفيذ **NexaLearn**، وهي منصة تعليم إلكتروني متكاملة مبنية بإطار عمل **NestJS** بلغة **TypeScript**، وفق مبادئ التصميم القائم على المجال (**DDD**) والهندسة النظيفية (**Clean Architecture**). يتكون النظام من أكثر من خمسة عشر سياقاً محدداً، من بينها: المصادقة، الدورات، التسجيل، المدفوعات (**Stripe**)، الوسائط (**Mux**)، خط أنابيب الذكاء الاصطناعي (توليد الاختبارات عبر **Whisper** ونماذج اللغة الكبيرة)، التقييم، المناقشات، التقديم، المراجعات، الشهادات، البحث، الاستمرارية، الإشعارات، وتحليلات المدربين.

يعتمد النهج المتبع على فصل أوامر **CQRS-lite**، وبنية موجهة بالأحداث، ونمط صندوق الصادر المعاملاتي (**Transactional Outbox**) لضمان تسليم الأحداث بموثوقية عالية. تُطبق المصادقة عبر **JWT** مع تدوير رموز التحديث، والتحكم في الوصول القائم على الأدوار (**RBAC**)، واستدعاءات موقعة **HMAC** بين الخدمات. تُستخدم **PostgreSQL** مع **Prisma ORM** للتخزين، و **Redis** للتخزين المؤقت وإدارة الجلسات، و **Stripe** للمدفوعات، و **Mux** لمعالجة الفيديو، و **AWS S3** للملفات، مع **Docker** للنشر.

أُنتج المشروع واجهة **REST API** شاملة تغطي دورة حياة التسجيل، وتتبع التقدم، ومحاولات الاختبارات، والمناقشات، وإصدار الشهادات، والبحث، ولوحات تحليلات المدربين. وتم التحقق من المسارات الحرجة عبر اختبارات تكامل باستخدام **Jest** و **Testcontainers**. يُظهر **NexaLearn** إمكانية بناء منصة تعليم إلكتروني حديثة قابلة للصيانة والاختبار والتوسع ضمن بنية **NestJS** منظمة.

Acknowledgment

We would like to express our sincere gratitude to our supervisor, Dr. Lydia Wahid, for the continuous guidance, constructive feedback, and academic support provided throughout every phase of this graduation project. Their expertise in software engineering and system architecture was instrumental in refining the design decisions, validating the architectural approach, and maintaining the technical rigor expected of a production-grade platform.

We are deeply indebted to the Faculty of Engineering at Cairo University, and particularly to the Department of Computer Engineering, for providing the academic environment, laboratory resources, and intellectual foundation that made this work possible. The curriculum and research culture of the department encouraged us to pursue a solution that is not merely functional, but also scalable, maintainable, and aligned with industry best practices.

Our appreciation extends to all faculty members and teaching assistants who contributed, directly or indirectly, to our education in software design, databases, distributed systems, and project management. We also thank our colleagues and peers whose technical discussions, code reviews, and collaborative debugging sessions helped improve the quality of the NexaLearn implementation.

Finally, we owe a profound debt of gratitude to our families and friends for their patience, encouragement, and unwavering support during the long hours of analysis, development, testing, and documentation required to complete this report.

List of Abbreviations

Abbreviation	Definition
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CAP	Consistency, Availability, Partition tolerance
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DRY	Don't Repeat Yourself
DXA	Domain eXperience Architecture
FK	Foreign Key
GP	Graduation Project
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model–View–Controller
OCC	Optimistic Concurrency Control
ORM	Object-Relational Mapping
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
S3	Simple Storage Service (Amazon S3)
SDK	Software Development Kit
SOLID	Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion
SQL	Structured Query Language
SSO	Single Sign-On
TTL	Time-To-Live
UUID	Universally Unique Identifier